

AI ASSISTANT CODING

ASSIGNMENT-11.5

Name: Amma Anjali

Enrollment no: 2303A52385

Batch: 33

Semester: VI

Branch: CSE(AIML)

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code: class

Stack:

pass

Expected Output:

- A functional stack implementation with all required methods and docstrings.

```
"""Prompt:
Generate a Python class named Stack that implements the stack data
structure using a list. The class should include the following
methods:
push(item) - to add an element to the top of the stack
pop() - to remove and return the top element
peek() - to return the top element without removing it
is_empty() - to check whether the stack is empty
Include proper docstrings for the class and each method. Handle edge
cases such as popping from an empty stack."""
```

```
main.py  Run  Output
10 class Stack:
11     """
12     A class to represent a stack data structure.
13     Uses a Python list to store stack elements.
14     """
15     def __init__(self):
16         """Initialize an empty stack."""
17         self.items = []
18
19     def push(self, item):
20         """
21         Add an element to the top of the stack.
22         param [int] item: Element to be added to the stack
23         """
24         self.items.append(item)
25
26     def pop(self):
27         """
28         Remove and return the top element of the stack.
29         :return: Top element of the stack
30         :raises IndexError: If the stack is empty
31         """
32         if self.is_empty():
33             raise IndexError("Pop from empty stack")
34         return self.items.pop()
35
36     def peek(self):
37         """
38         Return the top element without removing it.
39         :return: Top element of the stack
40         :raises IndexError: If the stack is empty
41         """
42         if self.is_empty():
43             raise IndexError("Peek from empty stack")
44         return self.items[-1]
45
46     def is_empty(self):
47         """
48         Check whether the stack is empty.
49         :return: True if stack is empty, False otherwise
50         """
51         return len(self.items) == 0
52
53 if __name__ == "__main__":
54     stack = Stack()
55     stack.push(10)
56     stack.push(20)
57     stack.push(30)
58     print("Top element:", stack.peek()) # 30
59     print("Popped element:", stack.pop()) # 30
60     print("Is stack empty?", stack.is_empty()) # False
```

```
Top element: 30
Popped element: 30
Is stack empty? False
Code Execution Successful
```

Task Description #2 – Queue Implementation Task:

Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```

""" Prompt:
Generate a Python class named Queue that implements a FIFO (First-In-First-Out) queue using a Python list. The class should include the following methods:

enqueue(item) - to add an element to the rear of the queue
dequeue() - to remove and return the front element
peek() - to return the front element without removing it
size() - to return the number of elements in the queue """

```

```

main.py
11 class Queue:
12     """
13     A class to represent a FIFO (First-In-First-Out) queue.
14     Uses a Python list to store queue elements.
15     """
16     def __init__(self):
17         """Initialize an empty queue."""
18         self.items = []
19
20     def enqueue(self, item):
21         """
22         Add an element to the rear of the queue.
23         (params item: Element to be added to the queue)
24         """
25         self.items.append(item)
26
27     def dequeue(self):
28         """
29         Remove and return the front element of the queue.
30         (returns: Front element of the queue)
31         (raises IndexError: If the queue is empty)
32         """
33         if self.is_empty():
34             raise IndexError("Dequeue from empty queue")
35         return self.items.pop(0)
36
37     def peek(self):
38         """
39         Return the front element without removing it.
40         (returns: Front element of the queue)
41         (raises IndexError: If the queue is empty)
42         """
43
44     def size(self):
45         """
46         Return the number of elements in the queue.
47         (returns: Number of elements in the queue)
48         """
49         return len(self.items)
50
51     def is_empty(self):
52         """
53         Check whether the queue is empty.
54         (returns: True if queue is empty, False otherwise)
55         """
56         return len(self.items) == 0
57
58 if __name__ == "__main__":
59     queue = Queue()
60
61     queue.enqueue(10)
62     queue.enqueue(20)
63     queue.enqueue(30)
64
65     print("Front element:", queue.peek()) # 10
66     print("Dequeued element:", queue.dequeue()) # 10
67     print("Queue size:", queue.size()) # 2

```

```

Output
Front element: 10
Dequeued element: 10
Queue size: 2
Code Execution Successful

```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

LinkedList:

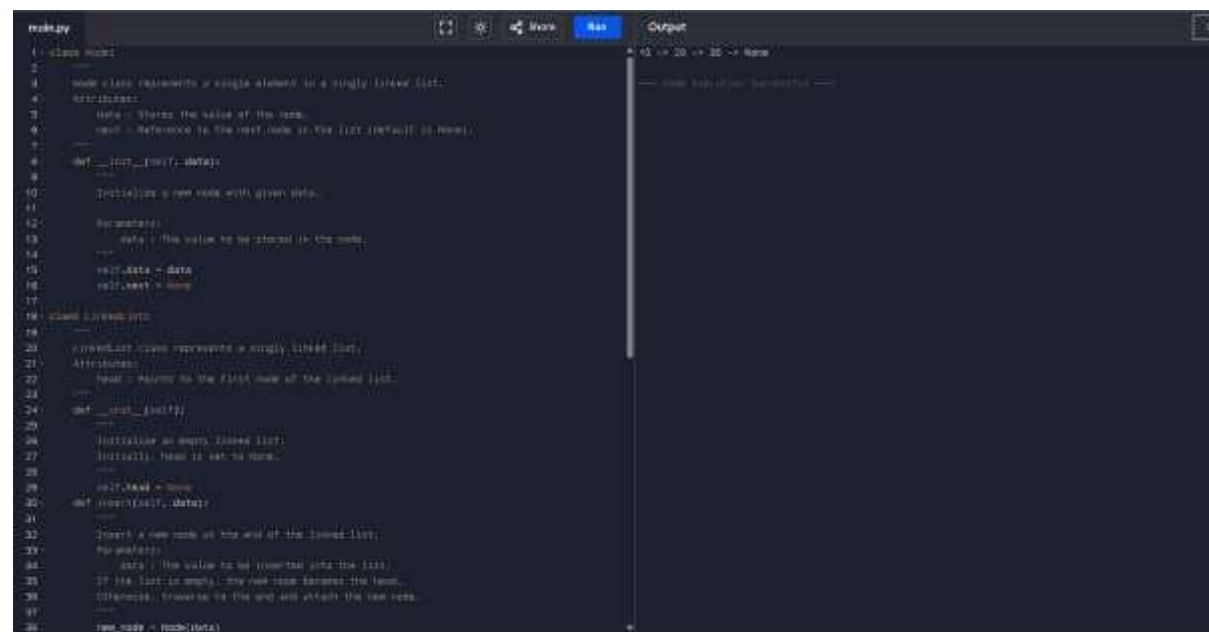
pass

Expected Output:

- A working linked list implementation with clear method documentation

```
""" Prompt:
Generate a Python implementation of a Singly Linked List.
Create a Node class to store data and a reference to the next node.
Create a LinkedList class with the following methods:

insert(data) - to insert a new node at the end of the list
display() - to display all elements of the linked list
Include clear docstrings for all classes and methods. """
```



```
1 class Node:
2     """
3     Node class represents a single element in a singly linked list.
4     Attributes:
5     data: Stores the value of the node.
6     next: Reference to the next node in the list (default is None).
7     """
8     def __init__(self, data):
9         """
10        Initialize a new node with given data.
11        """
12        self.data = data
13        self.next = None
14
15 class LinkedList:
16     """
17     LinkedList class represents a singly linked list.
18     Attributes:
19     head: Points to the first node of the linked list.
20     """
21     def __init__(self):
22         """
23         Initialize an empty linked list.
24         Initially, head is set to None.
25         """
26         self.head = None
27
28     def insert(self, data):
29         """
30        Insert a new node at the end of the linked list.
31        """
32        if self.head is None:
33            # If the list is empty, the new node becomes the head.
34            self.head = Node(data)
35        else:
36            # Traverse to the end and attach the new node.
37            current = self.head
38            while current.next is not None:
39                current = current.next
40            current.next = Node(data)
```

```

11: # If the list is empty, the new node becomes the head.
12: # Otherwise, traverse to the end and attach the new node.
13:
14: new_node = Node(data)
15: # If list is empty, there is no head.
16: if self.head is None:
17:     self.head = new_node
18:     return
19: # Traverse to the last node.
20: current = self.head
21: while current.next:
22:     current = current.next
23: # Link the new node to the last node.
24: current.next = new_node
25:
26: def display(self):
27:     """Display all elements of the linked list.
28:     Traverse from head to the last node
29:     and prints each element followed by ">".
30:     """
31:     current = self.head
32:     if current is None:
33:         print("Linked list is empty.")
34:         return
35:     while current:
36:         print(current.data, end="> ")
37:         current = current.next
38:
39:
40: if __name__ == "__main__":
41:     linked_list = LinkedList()
42:
43:     linked_list.insert(10)
44:     linked_list.insert(20)
45:     linked_list.insert(30)
46:     linked_list.display()

```

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code: class

HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods

```

1 """Task Description #4 – Hash Table
2 Task: Use AI to implement a hash table with basic insert, search, and delete
3 methods.
4 Sample Input Code:
5 class HashTable:
6     pass
7 Expected Output:
8 • Collision handling using chaining, with well-commented methods. """

```

```

1 class HashTable:
2     """
3     HashTable implementation using chaining for collision handling.
4     Attributes:
5     size: Size of the hash table.
6     table: List of buckets (each bucket is a list).
7     """
8     def __init__(self, size=10):
9         """
10        Initialize the hash table with given size.
11        Parameters:
12        size (int): Number of buckets in the hash table.
13        """
14        self.size = size
15        self.table = [[] for _ in range(size)]
16
17    def search(self, key):
18        """
19        Returns method to search key in the hash table.
20        Parameters:
21        key: Key to search.
22        Returns:
23        list: List of values associated with the key.
24        """
25        return self._search(key)
26
27    def insert(self, key, value):
28        """
29        Inserts a key-value pair into the hash table.
30        Handles collisions using chaining.
31        If the key already exists, updates its value.
32        """
33        index = self._hash(key)
34        bucket = self.table[index]
35        for pair in bucket:
36            if pair[0] == key:
37                pair[1] = value  # Update existing key
38            else:
39                bucket.append([key, value])  # Add new key-value pair
40
41    def _search(self, key):
42        """
43        Search for a key in the hash table.
44        Parameters:
45        key: Key to search.
46        Returns:
47        list: List of values associated with the key, or None if not found.
48        """
49        index = self._hash(key)
50        bucket = self.table[index]
51        for pair in bucket:
52            if pair[0] == key:
53                return pair[1]
54        return None  # Key not found
55
56    def _hash(self, key):
57        """
58        Returns a key-value pair from the hash table.
59        Parameters:
60        key: Key to search.
61        Returns:
62        bool: True if deleted successfully, False if key not found.
63        """
64        index = self._hash(key)
65        bucket = self.table[index]
66        for i, pair in enumerate(bucket):
67            if pair[0] == key:
68                del bucket[i]
69                return True
70        return False
71
72    def __str__(self):
73        """
74        String representation of the hash table.
75        """
76        return f"HashTable (Size: {self.size})\n"
77        for key, value in self.items():
78            print(f"Key: {key}, Value: {value}")
79
80    def __len__(self):
81        """
82        Returns the number of key-value pairs in the hash table.
83        """
84        return sum(len(bucket) for bucket in self.table)

```

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
```

```
pass
```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

```
- """ Prompt:
2 Generate a Python class named Graph that represents a graph using an adjacency list.
3 The class should include the following methods:
4 add_vertex(vertex) - to add a new vertex to the graph
5 add_edge(vertex1, vertex2) - to add an edge between two vertices
6 display() - to display all vertices and their connected neighbors. """

class Graph:
    """Generate an empty graph"""
    def __init__(self):
        self.vertices = {}
        self.edges = {}

    def add_vertex(self, vertex):
        """Add a vertex to the graph"""
        self.vertices[vertex] = []

    def add_edge(self, vertex1, vertex2):
        """Add an edge between two vertices"""
        self.vertices[vertex1].append(vertex2)
        self.vertices[vertex2].append(vertex1)

    def display(self):
        """Display all vertices and their connected neighbors"""
        for vertex in self.vertices:
            print(f"Vertex: {vertex}, Neighbors: {self.vertices[vertex]}")
```

Task Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue

o Priority

Queue o

Linked List

- o Binary Search Tree (BST)
- o Graph o Hash Table o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Smart Hospital Management System – Data Structure Selection

1. Patient Check-In System Selected

Data Structure: Queue

Justification:

A Queue follows the First-In-First-Out (FIFO) principle. Patients are treated in the order they arrive, ensuring fairness. Therefore, a Queue is the most suitable structure for managing regular patient check-ins.

2. Emergency Case Handling

Selected Data Structure: Priority Queue

Justification:

Emergency patients must be treated based on severity rather than arrival time. A Priority Queue allows higher-priority (critical) patients to be treated first. This ensures immediate attention to life-threatening cases.

3. Medical Records Storage Selected Data Structure: Hash

Table

Justification:

Patient records need fast retrieval using a unique patient ID. A Hash Table provides average $O(1)$ time complexity for search, insert, and delete operations. This makes record management efficient and scalable.

4. Doctor Appointment Scheduling

Selected Data Structure: Binary Search Tree (BST)

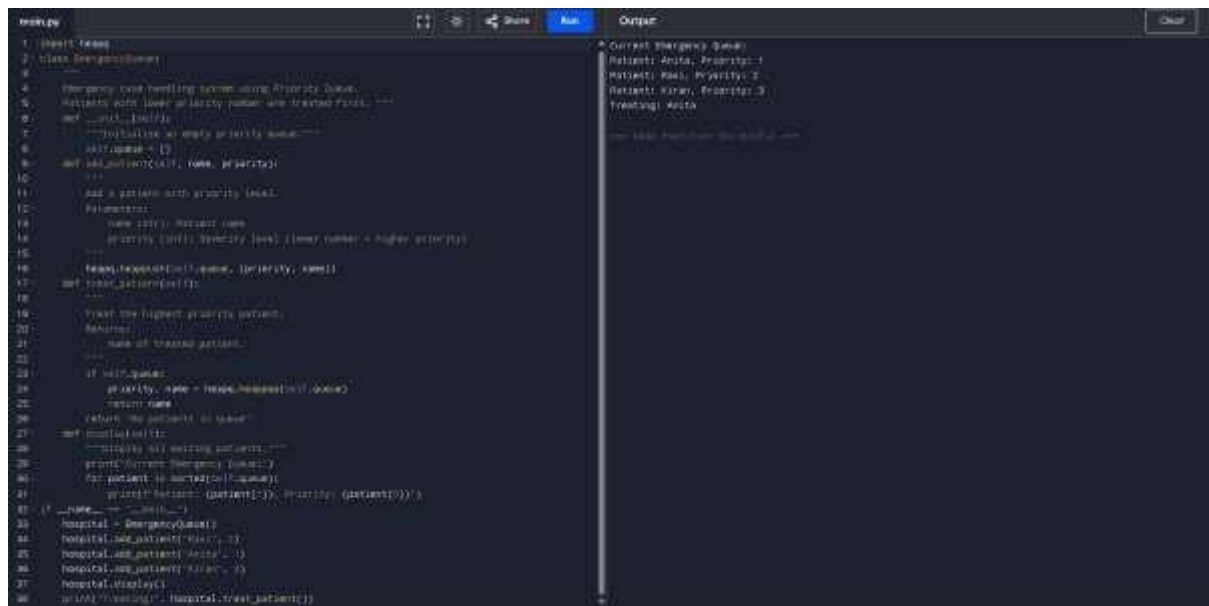
Justification:

Appointments must be maintained in sorted order based on time. A Binary Search Tree automatically keeps elements sorted. This makes scheduling and time-based retrieval efficient.

5. Hospital Room Navigation Selected Data Structure: Graph

Justification:

Hospital rooms and wards can be represented as nodes, and pathways as edges. A Graph effectively models connections between locations. It also supports pathfinding for navigation within the hospital.



```
1 // Queue Queue
2 class EmergencyQueue {
3   // Constructor
4   // Emergency queue handling system using Priority Queue.
5   // Patients with lower priority number are treated faster. ---
6   // ...
7   // Initialize an empty priority queue.
8   // ...
9   // ...
10  // ...
11  // Add a patient with priority level.
12  // ...
13  // ...
14  // ...
15  // ...
16  // ...
17  // ...
18  // ...
19  // ...
20  // ...
21  // ...
22  // ...
23  // ...
24  // ...
25  // ...
26  // ...
27  // ...
28  // ...
29  // ...
30  // ...
31  // ...
32  // ...
33  // ...
34  // ...
35  // ...
36  // ...
37  // ...
38  // ...
39  // ...
40  // ...
41  // ...
42  // ...
43  // ...
44  // ...
45  // ...
46  // ...
47  // ...
48  // ...
49  // ...
50  // ...
51  // ...
52  // ...
53  // ...
54  // ...
55  // ...
56  // ...
57  // ...
58  // ...
59  // ...
60  // ...
61  // ...
62  // ...
63  // ...
64  // ...
65  // ...
66  // ...
67  // ...
68  // ...
69  // ...
70  // ...
71  // ...
72  // ...
73  // ...
74  // ...
75  // ...
76  // ...
77  // ...
78  // ...
79  // ...
80  // ...
81  // ...
82  // ...
83  // ...
84  // ...
85  // ...
86  // ...
87  // ...
88  // ...
89  // ...
90  // ...
91  // ...
92  // ...
93  // ...
94  // ...
95  // ...
96  // ...
97  // ...
98  // ...
99  // ...
100 // ...
101 // ...
102 // ...
103 // ...
104 // ...
105 // ...
106 // ...
107 // ...
108 // ...
109 // ...
110 // ...
111 // ...
112 // ...
113 // ...
114 // ...
115 // ...
116 // ...
117 // ...
118 // ...
119 // ...
120 // ...
121 // ...
122 // ...
123 // ...
124 // ...
125 // ...
126 // ...
127 // ...
128 // ...
129 // ...
130 // ...
131 // ...
132 // ...
133 // ...
134 // ...
135 // ...
136 // ...
137 // ...
138 // ...
139 // ...
140 // ...
141 // ...
142 // ...
143 // ...
144 // ...
145 // ...
146 // ...
147 // ...
148 // ...
149 // ...
150 // ...
151 // ...
152 // ...
153 // ...
154 // ...
155 // ...
156 // ...
157 // ...
158 // ...
159 // ...
160 // ...
161 // ...
162 // ...
163 // ...
164 // ...
165 // ...
166 // ...
167 // ...
168 // ...
169 // ...
170 // ...
171 // ...
172 // ...
173 // ...
174 // ...
175 // ...
176 // ...
177 // ...
178 // ...
179 // ...
180 // ...
181 // ...
182 // ...
183 // ...
184 // ...
185 // ...
186 // ...
187 // ...
188 // ...
189 // ...
190 // ...
191 // ...
192 // ...
193 // ...
194 // ...
195 // ...
196 // ...
197 // ...
198 // ...
199 // ...
200 // ...
201 // ...
202 // ...
203 // ...
204 // ...
205 // ...
206 // ...
207 // ...
208 // ...
209 // ...
210 // ...
211 // ...
212 // ...
213 // ...
214 // ...
215 // ...
216 // ...
217 // ...
218 // ...
219 // ...
220 // ...
221 // ...
222 // ...
223 // ...
224 // ...
225 // ...
226 // ...
227 // ...
228 // ...
229 // ...
230 // ...
231 // ...
232 // ...
233 // ...
234 // ...
235 // ...
236 // ...
237 // ...
238 // ...
239 // ...
240 // ...
241 // ...
242 // ...
243 // ...
244 // ...
245 // ...
246 // ...
247 // ...
248 // ...
249 // ...
250 // ...
251 // ...
252 // ...
253 // ...
254 // ...
255 // ...
256 // ...
257 // ...
258 // ...
259 // ...
260 // ...
261 // ...
262 // ...
263 // ...
264 // ...
265 // ...
266 // ...
267 // ...
268 // ...
269 // ...
270 // ...
271 // ...
272 // ...
273 // ...
274 // ...
275 // ...
276 // ...
277 // ...
278 // ...
279 // ...
280 // ...
281 // ...
282 // ...
283 // ...
284 // ...
285 // ...
286 // ...
287 // ...
288 // ...
289 // ...
290 // ...
291 // ...
292 // ...
293 // ...
294 // ...
295 // ...
296 // ...
297 // ...
298 // ...
299 // ...
300 // ...
301 // ...
302 // ...
303 // ...
304 // ...
305 // ...
306 // ...
307 // ...
308 // ...
309 // ...
310 // ...
311 // ...
312 // ...
313 // ...
314 // ...
315 // ...
316 // ...
317 // ...
318 // ...
319 // ...
320 // ...
321 // ...
322 // ...
323 // ...
324 // ...
325 // ...
326 // ...
327 // ...
328 // ...
329 // ...
330 // ...
331 // ...
332 // ...
333 // ...
334 // ...
335 // ...
336 // ...
337 // ...
338 // ...
339 // ...
340 // ...
341 // ...
342 // ...
343 // ...
344 // ...
345 // ...
346 // ...
347 // ...
348 // ...
349 // ...
350 // ...
351 // ...
352 // ...
353 // ...
354 // ...
355 // ...
356 // ...
357 // ...
358 // ...
359 // ...
360 // ...
361 // ...
362 // ...
363 // ...
364 // ...
365 // ...
366 // ...
367 // ...
368 // ...
369 // ...
370 // ...
371 // ...
372 // ...
373 // ...
374 // ...
375 // ...
376 // ...
377 // ...
378 // ...
379 // ...
380 // ...
381 // ...
382 // ...
383 // ...
384 // ...
385 // ...
386 // ...
387 // ...
388 // ...
389 // ...
390 // ...
391 // ...
392 // ...
393 // ...
394 // ...
395 // ...
396 // ...
397 // ...
398 // ...
399 // ...
400 // ...
401 // ...
402 // ...
403 // ...
404 // ...
405 // ...
406 // ...
407 // ...
408 // ...
409 // ...
410 // ...
411 // ...
412 // ...
413 // ...
414 // ...
415 // ...
416 // ...
417 // ...
418 // ...
419 // ...
420 // ...
421 // ...
422 // ...
423 // ...
424 // ...
425 // ...
426 // ...
427 // ...
428 // ...
429 // ...
430 // ...
431 // ...
432 // ...
433 // ...
434 // ...
435 // ...
436 // ...
437 // ...
438 // ...
439 // ...
440 // ...
441 // ...
442 // ...
443 // ...
444 // ...
445 // ...
446 // ...
447 // ...
448 // ...
449 // ...
450 // ...
451 // ...
452 // ...
453 // ...
454 // ...
455 // ...
456 // ...
457 // ...
458 // ...
459 // ...
460 // ...
461 // ...
462 // ...
463 // ...
464 // ...
465 // ...
466 // ...
467 // ...
468 // ...
469 // ...
470 // ...
471 // ...
472 // ...
473 // ...
474 // ...
475 // ...
476 // ...
477 // ...
478 // ...
479 // ...
480 // ...
481 // ...
482 // ...
483 // ...
484 // ...
485 // ...
486 // ...
487 // ...
488 // ...
489 // ...
490 // ...
491 // ...
492 // ...
493 // ...
494 // ...
495 // ...
496 // ...
497 // ...
498 // ...
499 // ...
500 // ...
501 // ...
502 // ...
503 // ...
504 // ...
505 // ...
506 // ...
507 // ...
508 // ...
509 // ...
510 // ...
511 // ...
512 // ...
513 // ...
514 // ...
515 // ...
516 // ...
517 // ...
518 // ...
519 // ...
520 // ...
521 // ...
522 // ...
523 // ...
524 // ...
525 // ...
526 // ...
527 // ...
528 // ...
529 // ...
530 // ...
531 // ...
532 // ...
533 // ...
534 // ...
535 // ...
536 // ...
537 // ...
538 // ...
539 // ...
540 // ...
541 // ...
542 // ...
543 // ...
544 // ...
545 // ...
546 // ...
547 // ...
548 // ...
549 // ...
550 // ...
551 // ...
552 // ...
553 // ...
554 // ...
555 // ...
556 // ...
557 // ...
558 // ...
559 // ...
560 // ...
561 // ...
562 // ...
563 // ...
564 // ...
565 // ...
566 // ...
567 // ...
568 // ...
569 // ...
570 // ...
571 // ...
572 // ...
573 // ...
574 // ...
575 // ...
576 // ...
577 // ...
578 // ...
579 // ...
580 // ...
581 // ...
582 // ...
583 // ...
584 // ...
585 // ...
586 // ...
587 // ...
588 // ...
589 // ...
590 // ...
591 // ...
592 // ...
593 // ...
594 // ...
595 // ...
596 // ...
597 // ...
598 // ...
599 // ...
600 // ...
601 // ...
602 // ...
603 // ...
604 // ...
605 // ...
606 // ...
607 // ...
608 // ...
609 // ...
610 // ...
611 // ...
612 // ...
613 // ...
614 // ...
615 // ...
616 // ...
617 // ...
618 // ...
619 // ...
620 // ...
621 // ...
622 // ...
623 // ...
624 // ...
625 // ...
626 // ...
627 // ...
628 // ...
629 // ...
630 // ...
631 // ...
632 // ...
633 // ...
634 // ...
635 // ...
636 // ...
637 // ...
638 // ...
639 // ...
640 // ...
641 // ...
642 // ...
643 // ...
644 // ...
645 // ...
646 // ...
647 // ...
648 // ...
649 // ...
650 // ...
651 // ...
652 // ...
653 // ...
654 // ...
655 // ...
656 // ...
657 // ...
658 // ...
659 // ...
660 // ...
661 // ...
662 // ...
663 // ...
664 // ...
665 // ...
666 // ...
667 // ...
668 // ...
669 // ...
670 // ...
671 // ...
672 // ...
673 // ...
674 // ...
675 // ...
676 // ...
677 // ...
678 // ...
679 // ...
680 // ...
681 // ...
682 // ...
683 // ...
684 // ...
685 // ...
686 // ...
687 // ...
688 // ...
689 // ...
690 // ...
691 // ...
692 // ...
693 // ...
694 // ...
695 // ...
696 // ...
697 // ...
698 // ...
699 // ...
700 // ...
701 // ...
702 // ...
703 // ...
704 // ...
705 // ...
706 // ...
707 // ...
708 // ...
709 // ...
710 // ...
711 // ...
712 // ...
713 // ...
714 // ...
715 // ...
716 // ...
717 // ...
718 // ...
719 // ...
720 // ...
721 // ...
722 // ...
723 // ...
724 // ...
725 // ...
726 // ...
727 // ...
728 // ...
729 // ...
730 // ...
731 // ...
732 // ...
733 // ...
734 // ...
735 // ...
736 // ...
737 // ...
738 // ...
739 // ...
740 // ...
741 // ...
742 // ...
743 // ...
744 // ...
745 // ...
746 // ...
747 // ...
748 // ...
749 // ...
750 // ...
751 // ...
752 // ...
753 // ...
754 // ...
755 // ...
756 // ...
757 // ...
758 // ...
759 // ...
760 // ...
761 // ...
762 // ...
763 // ...
764 // ...
765 // ...
766 // ...
767 // ...
768 // ...
769 // ...
770 // ...
771 // ...
772 // ...
773 // ...
774 // ...
775 // ...
776 // ...
777 // ...
778 // ...
779 // ...
780 // ...
781 // ...
782 // ...
783 // ...
784 // ...
785 // ...
786 // ...
787 // ...
788 // ...
789 // ...
790 // ...
791 // ...
792 // ...
793 // ...
794 // ...
795 // ...
796 // ...
797 // ...
798 // ...
799 // ...
800 // ...
801 // ...
802 // ...
803 // ...
804 // ...
805 // ...
806 // ...
807 // ...
808 // ...
809 // ...
810 // ...
811 // ...
812 // ...
813 // ...
814 // ...
815 // ...
816 // ...
817 // ...
818 // ...
819 // ...
820 // ...
821 // ...
822 // ...
823 // ...
824 // ...
825 // ...
826 // ...
827 // ...
828 // ...
829 // ...
830 // ...
831 // ...
832 // ...
833 // ...
834 // ...
835 // ...
836 // ...
837 // ...
838 // ...
839 // ...
840 // ...
841 // ...
842 // ...
843 // ...
844 // ...
845 // ...
846 // ...
847 // ...
848 // ...
849 // ...
850 // ...
851 // ...
852 // ...
853 // ...
854 // ...
855 // ...
856 // ...
857 // ...
858 // ...
859 // ...
860 // ...
861 // ...
862 // ...
863 // ...
864 // ...
865 // ...
866 // ...
867 // ...
868 // ...
869 // ...
870 // ...
871 // ...
872 // ...
873 // ...
874 // ...
875 // ...
876 // ...
877 // ...
878 // ...
879 // ...
880 // ...
881 // ...
882 // ...
883 // ...
884 // ...
885 // ...
886 // ...
887 // ...
888 // ...
889 // ...
890 // ...
891 // ...
892 // ...
893 // ...
894 // ...
895 // ...
896 // ...
897 // ...
898 // ...
899 // ...
900 // ...
901 // ...
902 // ...
903 // ...
904 // ...
905 // ...
906 // ...
907 // ...
908 // ...
909 // ...
910 // ...
911 // ...
912 // ...
913 // ...
914 // ...
915 // ...
916 // ...
917 // ...
918 // ...
919 // ...
920 // ...
921 // ...
922 // ...
923 // ...
924 // ...
925 // ...
926 // ...
927 // ...
928 // ...
929 // ...
930 // ...
931 // ...
932 // ...
933 // ...
934 // ...
935 // ...
936 // ...
937 // ...
938 // ...
939 // ...
940 // ...
941 // ...
942 // ...
943 // ...
944 // ...
945 // ...
946 // ...
947 // ...
948 // ...
949 // ...
950 // ...
951 // ...
952 // ...
953 // ...
954 // ...
955 // ...
956 // ...
957 // ...
958 // ...
959 // ...
960 // ...
961 // ...
962 // ...
963 // ...
964 // ...
965 // ...
966 // ...
967 // ...
968 // ...
969 // ...
970 // ...
971 // ...
972 // ...
973 // ...
974 // ...
975 // ...
976 // ...
977 // ...
978 // ...
979 // ...
980 // ...
981 // ...
982 // ...
983 // ...
984 // ...
985 // ...
986 // ...
987 // ...
988 // ...
989 // ...
990 // ...
991 // ...
992 // ...
993 // ...
994 // ...
995 // ...
996 // ...
997 // ...
998 // ...
999 // ...
1000 // ...
```

Task Description #7: Smart City Traffic Control System A city

plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue

o Priority Queue o

Linked List

o Binary Search

Tree (BST) o Graph o

Hash Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Smart City Traffic Control System

1. Traffic Signal Queue Selected Data

Structure: Queue

Justification:

Vehicles waiting at a traffic signal follow the First-In-First-Out (FIFO) principle. The vehicle that arrives first should move first when the signal turns green. Therefore, a Queue is the most appropriate structure for managing traffic signals fairly and efficiently.

2. Emergency Vehicle Priority Handling

Selected Data Structure: Priority Queue

Justification:

Emergency vehicles such as ambulances and fire trucks must be given immediate access regardless of arrival time. A Priority Queue allows higher-priority vehicles to be

processed before regular vehicles. This ensures faster emergency response and improved public safety.

3. Vehicle Registration Lookup

Selected Data Structure: Hash Table

Justification:

Vehicle details must be accessed instantly using a registration number. A Hash Table provides average $O(1)$ time complexity for search operations. This makes it ideal for fast and efficient vehicle information retrieval.

4. Road Network Mapping Selected

Data Structure: Graph

Justification:

Roads and intersections can be represented as nodes and edges. A Graph efficiently models connections between locations and supports route-finding algorithms. This makes it suitable for navigation and traffic management systems.

5. Parking Slot Availability

Selected Data Structure: Binary Search Tree (BST)

Justification:

Parking slots can be stored in sorted order based on slot numbers. A Binary Search Tree maintains elements in sorted order and allows efficient insertion, deletion, and searching. This makes it suitable for tracking available and occupied parking slots.

```

main.py
1: class Stack:
2:     """
3:     Stack implementation using Python list.
4:     follows LIFO (Last-In-First-Out) principle. """
5:     def __init__(self):
6:         """
7:         Initialize an empty stack.
8:         """
9:         self._items = []
10:     def push(self, item):
11:         """
12:         Add an element to the top of the stack.
13:         Parameters:
14:         item (Element): Element to be added
15:         """
16:         self._items.append(item)
17:     def pop(self):
18:         """
19:         Remove and return the top element of the stack.
20:         Returns:
21:         The top element of the stack.
22:         Raises:
23:         IndexError: If stack is empty. """
24:         if self.is_empty():
25:             raise IndexError("Pop from empty stack")
26:         return self._items.pop()
27:     def peek(self):
28:         """
29:         Return the top element without removing it.
30:         Returns:
31:         The top element of the stack.
32:         Raises:
33:         IndexError: If stack is empty. """
34:
35:
36:
37:     def is_empty(self):
38:         """
39:         Check whether the stack is empty.
40:         Returns:
41:         True if stack is empty, otherwise False.
42:         """
43:         return len(self._items) == 0
44:     def size(self):
45:         """
46:         Return the number of elements in the stack.
47:         """
48:         return len(self._items)
49: if __name__ == "__main__":
50:     stack = Stack()
51:
52:     stack.push(10)
53:     stack.push(20)
54:     stack.push(30)
55:
56:     print("Top element:", stack.peek())
57:     print("Popped element:", stack.pop())
58:     print("Stack size:", stack.size())
59:     print("Is stack empty?", stack.is_empty())

```

```

Output
* Top element: 30
* Popped element: 30
* Stack size: 2
* Is stack empty? False

--- Sub-Interpreter Terminated ---

```

```

Output
* Top element: 30
* Popped element: 30
* Stack size: 2
* Is stack empty? False

--- Sub-Interpreter Terminated ---

```

Task Description #8: Smart E-Commerce Platform – Data Structure.

Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are Placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue

o Priority Queue o

Linked List

o Binary Search

Tree (BST) o Graph o

Hash Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Smart E-Commerce Platform – Data Structure Selection

1. Shopping Cart Management Selected

Data Structure: Linked List

Justification:

Products in a shopping cart are added and removed dynamically. A Linked List allows efficient insertion and deletion without shifting elements. This makes it suitable for managing frequently changing cart items.

2. Order Processing System Selected

Data Structure: Queue

Justification:

Orders must be processed in the order they are placed. A Queue follows the First-InFirst-Out (FIFO) principle, ensuring fairness. Therefore, it is ideal for handling customer orders.

3. Top-Selling Products Tracker

Selected Data Structure: Priority

Queue

Justification:

Products need to be ranked based on sales count. A Priority Queue allows items with higher sales to be accessed first. This ensures efficient tracking of top-selling products.

4. Product Search Engine

Selected Data Structure: Hash Table

Justification:

Products are searched using unique product IDs. A Hash Table provides average $O(1)$ time complexity for search operations. This makes product lookup fast and efficient.

5. Delivery Route Planning Selected

Data Structure: Graph

Justification:

Warehouses and delivery locations can be represented as nodes, and roads as edges. A Graph efficiently models connections and supports shortest path algorithms. This makes it suitable for route optimization and logistics planning.

```

1 # Create a function to add a product
2 # Function: Product Name, Price, Stock
3 # Return: None
4 # Parameters: product_name, price, stock
5 # Example: add_product('Laptop', 999, 10)
6
7 def add_product(product_name, price, stock):
8     products[product_id] = {
9         'name': product_name,
10         'price': price,
11         'stock': stock
12     }
13     print(f'Product added successfully.')
14
15 # Function to search for a product
16 # Parameters: product_id
17 # Return: product details or None
18
19 def search_product(product_id):
20     if product_id in products:
21         product = products[product_id]
22         print(f'Product ID: {product_id}')
23         print(f'Name: {product['name']}')
24         print(f'Price: {product['price']}')
25         print(f'Stock: {product['stock']}')
26     else:
27         print(f'Product not found.')
28
29 # Function to display all products
30 # Return: None
31
32 def display_products():
33     if len(products) == 0:
34         print('No products available.')
35     else:
36         print('Available Products:')
37         for product_id, product in products.items():
38             print(f'ID: {product_id} | Name: {product['name']} | Price: {product['price']} | Stock: {product['stock']}')
39
40 # Main Function
41 # Add products
42 add_product('Laptop', 999, 10)
43 add_product('Mobile', 2000, 5)
44 add_product('Headphones', 100, 20)
45
46 # Display products
47 display_products()
48
49 # Search for Product 101
50 search_product(101)
51
52 # Search for Product 200
53 search_product(200)
54
55 # Search for Product 102
56 search_product(102)

```

```

Product added successfully.
Product added successfully.
Product added successfully.

Available Products:
ID: 1 | Name: Laptop | Price: 999
ID: 2 | Name: Mobile | Price: 2000
ID: 3 | Name: Headphones | Price: 100

Searching for Product 101

Product Found:
Product ID: 101
Name: Mobile
Price: 2000

Searching for Product 200

Product not found.

Searching for Product 102

```